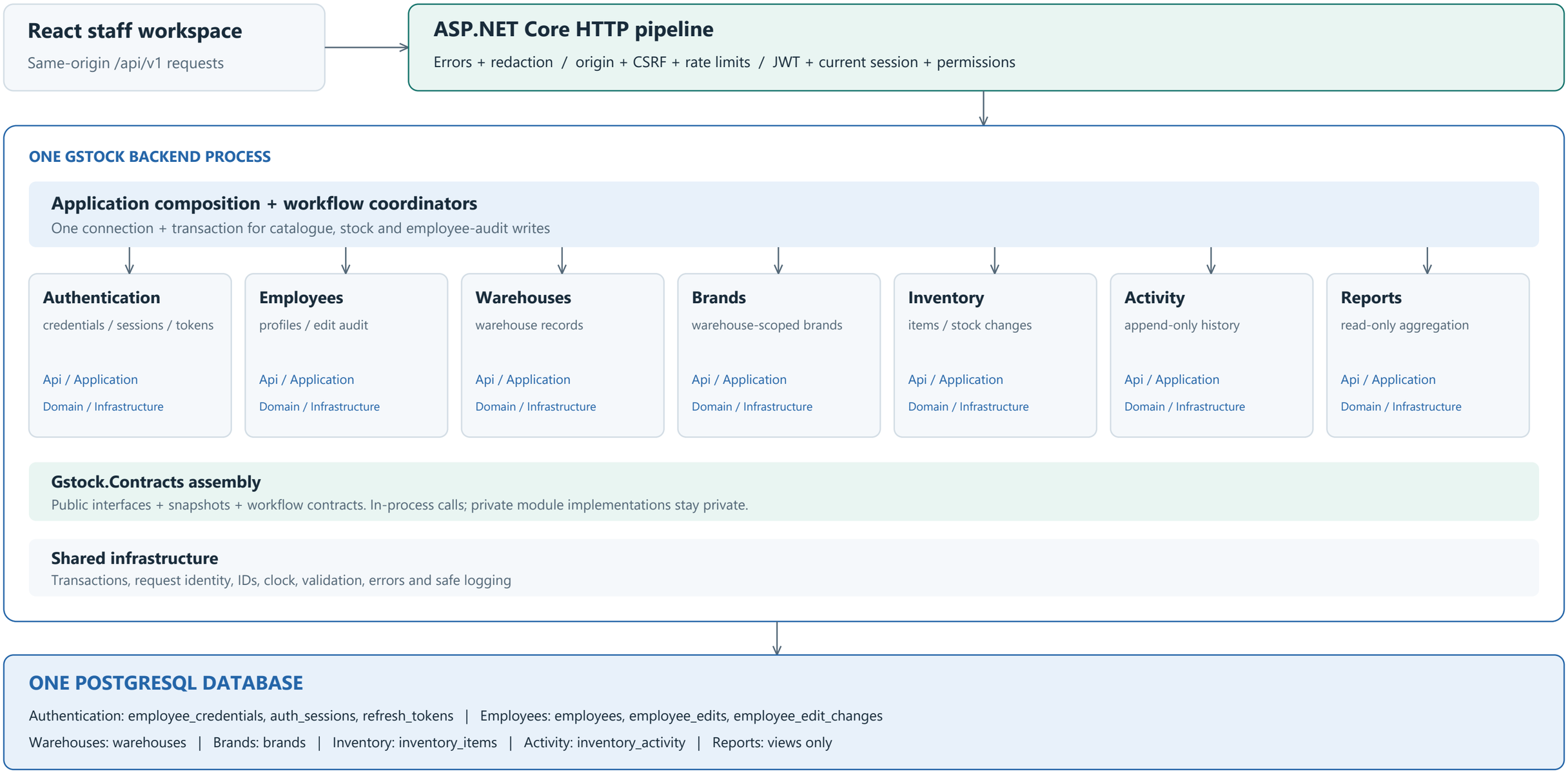
One application. Seven modules.

Runtime architecture, module ownership and the rules that keep the monolith modular.



Boundary rules

Only the owner module writes a table. Cross-module calls use contracts. Published SQL views allow approved read models; Reports uses a SELECT-only connection. Coordinators commit business writes and Activity together.

Complete file map: pages 2-8 / 248 proposed files / folder and document symbols distinguish node types

Repository, host and shared infrastructure

The backend sits alongside the existing frontend; these files describe the proposed implementation baseline.

Folder/ File.cs Lines show containment

Repository & deployment

Paths relative to repository root

- .github/
  - workflows/
    - backend-ci.yml
- backend/
  - Gstock.Backend.slnx
  - global.json
  - .editorconfig
  - .gitignore
  - .dockerignore
  - .env.example
  - README.md
  - Dockerfile
  - compose.yaml
  - deploy/
    - nginx.conf
  - docs/
    - openapi.yaml
    - module-boundaries.md
    - operations.md

One backend image. PostgreSQL and a same-origin reverse proxy are deployment infrastructure. Secrets and signing keys are supplied externally.

Application host

backend/src/Gstock.Backend/

- Gstock.Backend.csproj
- Program.cs
- appsettings.json
- appsettings.Development.json
- Properties/
  - launchSettings.json
- App/
  - Composition/
    - ServiceRegistration.cs
    - ModuleRegistration.cs
  - Configuration/
    - DatabaseOptions.cs
    - AuthOptions.cs
    - OriginOptions.cs
  - Http/
    - ExceptionHandler.cs
    - RequestIdentityMiddleware.cs
    - RequestLoggingMiddleware.cs
    - OriginValidationMiddleware.cs
    - EndpointPolicies.cs
    - AuthenticationSetup.cs
    - CsrfEndpointFilter.cs
    - RateLimitingSetup.cs
    - HealthEndpoints.cs
  - Coordinators/
    - CatalogueCoordinator.cs
    - InventoryCoordinator.cs
    - EmployeeCoordinator.cs

Program wires one ASP.NET Core host. Coordinators implement workflow contracts and own transactions for cross-module writes. Public health probes reveal no secrets.

Shared infrastructure

backend/src/Gstock.Backend/Shared/

- Persistence/
  - PostgresConnectionFactory.cs
  - PostgresUnitOfWork.cs
  - DatabaseErrorMapper.cs
  - SqlReadExecutor.cs
- Identity/
  - CurrentRequestIdentity.cs
- Time/
  - SystemClock.cs
- Ids/
  - UuidGenerator.cs
- Validation/
  - TextNormalization.cs
  - PaginationRules.cs
- Errors/
  - ApplicationException.cs
  - ErrorResponse.cs
- Observability/
  - LogRedaction.cs

Use Npgsql with parameterized SQL. A unit of work carries the same connection and transaction to participating repositories; read executors accept explicit published projections.

Public contracts, migrations and administration

Gstock.Backend references Gstock.Contracts. Contracts never reference the host or module implementations.

Folder/ File.cs Lines show containment

Public contracts

backend/src/Gstock.Contracts/

- Gstock.Contracts.csproj
- Common/
  - IUnitOfWork.cs
  - IClock.cs
  - IIdGenerator.cs
  - IRequestIdentity.cs
  - ActorSnapshot.cs
  - PageRequest.cs
  - PageResult.cs
  - ErrorCodes.cs
- Authentication/
  - ICredentialService.cs
  - ISessionValidator.cs
  - IAuthSessionMaintenance.cs
- Employees/
  - IEmployeeDirectory.cs
  - IEmployeeEditor.cs
  - IEmployeeAdministration.cs
  - EmployeeSnapshot.cs
  - EmployeeEditCommand.cs
  - EmployeeEditResult.cs
- Warehouses/
  - IWarehouseDirectory.cs
  - IWarehouseWriter.cs
  - WarehouseSnapshot.cs
  - WarehouseCreateCommand.cs
- Brands/
  - IBrandDirectory.cs
  - IBrandWriter.cs
  - BrandSnapshot.cs
  - BrandCreateCommand.cs
- Inventory/
  - IInventoryService.cs
  - InventorySnapshot.cs
  - InventoryWriteCommands.cs
- Activity/
  - IActivityRecorder.cs
  - ActivityRecord.cs
- Reports/
  - IStockReportReader.cs
  - StockReportContracts.cs
- Workflows/
  - ICatalogueWorkflow.cs
  - IInventoryWorkflow.cs
  - IEmployeeWorkflow.cs

A separate assembly contains interfaces and data records only. Modules depend on it, never on another module's private repositories. Grouped DTO files contain related named records.

Database & administration

backend/

- migrations/
  - 001_employees.sql
  - 002_authentication.sql
  - 003_warehouses.sql
  - 004_brands.sql
  - 005_inventory_items.sql
  - 006_inventory_activity.sql
  - 007_employee_audit.sql
  - 008_published_read_views.sql
  - 009_database_roles.sql
- src/
  - Gstock.Backend/
    - App/
      - Administration/
        - AdminCommandRunner.cs
        - MigrateDatabaseCommand.cs
        - ProvisionEmployeeCommand.cs
        - ChangeAccountStatusCommand.cs
        - CleanupAuthSessionsCommand.cs

Management mode runs the same assembly without starting HTTP. Migrations use a privileged connection. Runtime and report-reader roles have restricted grants; migration history is tool-managed.

Authentication and employees

All modules have explicit Api, Application, Domain and Infrastructure folders; each owns its persistence adapters.

Folder/ File.cs Lines show containment

Authentication

backend/src/Gstock.Backend/Modules/Authentication/

- AuthenticationModule.cs
- Api/
  - AuthController.cs
  - AuthDtos.cs
  - AuthRequestValidator.cs
  - RefreshCookieWriter.cs
- Application/
  - AuthService.cs
  - RefreshTokenService.cs
  - CredentialService.cs
  - SessionValidator.cs
  - SessionCleanupService.cs
  - ICredentialRepository.cs
  - ISessionRepository.cs
  - IRefreshTokenRepository.cs
  - IPasswordHasher.cs
  - IAccessTokenService.cs
  - IRefreshTokenGenerator.cs
  - ICsrfTokenService.cs
- Domain/
  - EmployeeCredential.cs
  - AuthSession.cs
  - RefreshToken.cs
  - SessionLifetimePolicy.cs
  - PasswordPolicy.cs
- Infrastructure/
  - PostgresCredentialRepository.cs
  - PostgresSessionRepository.cs
  - PostgresRefreshTokenRepository.cs
  - Argon2idPasswordHasher.cs
  - JwtAccessTokenService.cs
  - JwtBearerHandler.cs
  - SigningKeyProvider.cs
  - RefreshTokenGenerator.cs
  - SignedCsrfTokenService.cs

Owns employee_credentials, auth_sessions and refresh_tokens. Verifies active employee/session state on protected requests; refresh locks the family and commits replay revocation before returning 401.

Employees

backend/src/Gstock.Backend/Modules/Employees/

- EmployeesModule.cs
- Api/
  - EmployeesController.cs
  - EmployeeDtos.cs
  - EmployeeRequestValidator.cs
  - EmployeeResponseMapper.cs
- Application/
  - EmployeeQueryService.cs
  - EmployeeDirectory.cs
  - EmployeeEditor.cs
  - EmployeeAdministrationService.cs
  - IEmployeeRepository.cs
  - IEmployeeEditRepository.cs
- Domain/
  - Employee.cs
  - EmployeeEdit.cs
  - EmployeeEditChange.cs
  - EmployeeRules.cs
- Infrastructure/
  - PostgresEmployeeRepository.cs
  - PostgresEmployeeEditRepository.cs

Owns employees, employee_edits and employee_edit_changes. Profile writes require current management permission; passwords and hashes never enter response or audit DTOs.

Warehouses and brands

Catalogue creation calls public writer contracts and records the matching Activity event in the same unit of work.

Folder/ File.cs Lines show containment

Warehouses

backend/src/Gstock.Backend/Modules/Warehouses/

- WarehousesModule.cs
- Api/
  - WarehousesController.cs
  - WarehouseDtos.cs
  - WarehouseRequestValidator.cs
  - WarehouseResponseMapper.cs
- Application/
  - WarehouseQueryService.cs
  - WarehouseDirectory.cs
  - WarehouseWriter.cs
  - IWarehouseRepository.cs
- Domain/
  - Warehouse.cs
  - WarehouseRules.cs
- Infrastructure/
  - PostgresWarehouseRepository.cs
  - Projections/
    - WarehouseDirectoryView.sql

Owns warehouses. Creation goes through CatalogueCoordinator and appends Activity in the same transaction. Count queries use approved Inventory and Brands views.

Brands

backend/src/Gstock.Backend/Modules/Brands/

- BrandsModule.cs
- Api/
  - BrandsController.cs
  - BrandDtos.cs
  - BrandRequestValidator.cs
  - BrandResponseMapper.cs
- Application/
  - BrandQueryService.cs
  - BrandDirectory.cs
  - BrandWriter.cs
  - IBrandRepository.cs
- Domain/
  - Brand.cs
  - BrandRules.cs
- Infrastructure/
  - PostgresBrandRepository.cs
  - Projections/
    - BrandDirectoryView.sql

Owns brands. Warehouse identity comes from IWarehouseDirectory. Names are unique within a warehouse; initials and stock-attention counts are derived for responses.

Inventory and activity

The InventoryCoordinator owns the transaction: lock + compare + update stock + append history + commit.

Folder/ File.cs Lines show containment

Inventory

backend/src/Gstock.Backend/Modules/Inventory/

- InventoryModule.cs
- Api/
  - ItemsController.cs
  - InventorySummaryController.cs
  - InventoryDtos.cs
  - InventoryRequestValidator.cs
  - InventoryResponseMapper.cs
- Application/
  - InventoryQueryService.cs
  - InventoryService.cs
  - IInventoryRepository.cs
- Domain/
  - InventoryItem.cs
  - StockAdjustment.cs
  - StockStatus.cs
  - StockUnit.cs
  - InventoryRules.cs
- Infrastructure/
  - PostgresInventoryRepository.cs
  - Projections/
    - InventoryCatalogueView.sql

Owns inventory_items. Derives warehouse from brand. Lock item, compare expectedQuantity, validate bounds, and append Activity before committing. Stock status is derived.

Activity

backend/src/Gstock.Backend/Modules/Activity/

- ActivityModule.cs
- Api/
  - ActivityController.cs
  - ActivityDtos.cs
  - ActivityRequestValidator.cs
  - ActivityResponseMapper.cs
- Application/
  - ActivityQueryService.cs
  - ActivityRecorder.cs
  - IActivityRepository.cs
- Domain/
  - InventoryActivity.cs
  - ActivityType.cs
  - ActivityRules.cs
- Infrastructure/
  - PostgresActivityRepository.cs
  - Projections/
    - StockMovementView.sql

Owns append-only inventory_activity. Server identity supplies actor and timestamp snapshots. Module APIs expose append/read operations; history has no ordinary update or delete path.

Reports and verification

Read-only reporting uses published projections; automated boundary checks guard the modular monolith structure.

Folder/ File.cs Lines show containment

Reports

backend/src/Gstock.Backend/Modules/Reports/

- ReportsModule.cs
- Api/
  - ReportsController.cs
  - ReportDtos.cs
  - ReportRequestValidator.cs
  - ReportResponseMapper.cs
- Application/
  - StockReportService.cs
  - IStockMovementReader.cs
- Domain/
  - ReportWindow.cs
  - ReportBucketPolicy.cs
  - StockMovementTotals.cs
- Infrastructure/
  - PostgresStockMovementReader.cs

Owns no authoritative tables. A separate SELECT-only connection reads published views. Bucket in Africa/Johannesburg, fill gaps with zeros, and keep each report to one stock unit.

Unit & architecture tests

backend/tests/

- Gstock.UnitTests/
  - Gstock.UnitTests.csproj
  - Authentication/
    - PasswordPolicyTests.cs
    - SessionLifetimeTests.cs
  - Employees/
    - EmployeeEditRulesTests.cs
  - Inventory/
    - StockAdjustmentTests.cs
    - StockStatusTests.cs
  - Activity/
    - ActivityRulesTests.cs
  - Reports/
    - StockReportBucketTests.cs
    - StockMovementTotalsTests.cs
- Gstock.ArchitectureTests/
  - Gstock.ArchitectureTests.csproj
  - ModuleBoundaryTests.cs
  - LayerDependencyTests.cs
  - PublishedReadAccessTests.cs

Pure domain checks cover rules and calculations. Architecture checks enforce module ownership, layer dependencies and approved read access.

Integration checks and atomic workflows

Real PostgreSQL checks cover constraints, concurrency, revocation, audit atomicity and unit-separated reports.

Folder/ File.cs Lines show containment

Integration tests

backend/tests/

- Gstock.IntegrationTests/
  - Gstock.IntegrationTests.csproj
  - Fixtures/
    - GstockApiFactory.cs
    - PostgresFixture.cs
    - TestIdentity.cs
  - Authentication/
    - PasswordHashingTests.cs
    - JwtValidationTests.cs
    - SessionRevocationTests.cs
    - RefreshRotationTests.cs
    - RefreshConcurrencyTests.cs
    - CookieCsrfRateLimitTests.cs
    - CredentialRedactionTests.cs
  - Employees/
    - EmployeePermissionAuditTests.cs
    - PasswordChangeAtomicityTests.cs
    - AccountStatusConcurrencyTests.cs
  - Warehouses/
    - WarehouseQueryCreateTests.cs
  - Brands/
    - BrandScopeQueryCreateTests.cs
  - Inventory/
    - ItemQueryCreateTests.cs
    - InventoryConstraintTests.cs
    - StockConcurrencyTests.cs
    - ActivityRollbackTests.cs
  - Activity/
    - ActivityHistoryTests.cs
  - Reports/
    - StockReportQueryTests.cs
  - Database/
    - MigrationAndViewTests.cs

These are planned checks, not claims of passing tests. Integration tests use real PostgreSQL semantics for locks, constraints, rollback, refresh replay and read-only grants.

Stock change

1 Authenticate + authorize
Reject before business work on failure.

↓

2 Begin one transaction
One connection passed to both modules.

↓

3 Lock + compare + update
Inventory validates expectedQuantity.

↓

4 Append Activity
Capture actor, old/new values and reason.

↓

5 Commit once
Any earlier failure rolls back both writes.

Password change

EmployeeCoordinator locks employee then sessions, saves the profile and edit record, calls ICredentialService to hash/change the credential and revoke all families, then commits everything together.

Refresh-token replay

RefreshTokenService detects a consumed token, revokes its whole session family, commits that revocation, and only then returns 401. A generic exception rollback must not undo this security decision.

Verification files are proposed; no backend tests were run.